Task 2 — Rolling Chassis

The rover, and driving it

Task 2

https://github.com/KrithinP/vanguard-inductions-2026

Every link in this document opens the live page on GitHub.

PART 1

Your mission

Build a four-wheel rover and drive it

TASK 2 — "Rolling Chassis"

> **Objective:** build the rover, put it in the world, and drive it. **Effort:** 6–8 hours.

Mission briefing

The flight computer is awake. It has no idea it's attached to a rover.

Before software can drive a vehicle it needs to know what the vehicle *is* — how long the chassis is, where the wheels sit, which way is forward. That description is a real file, and every part of the stack reads it: the simulator to build a body, the visualiser to draw it, the navigation stack to know how wide a gap it can fit through.

This Sol you describe the rover, spawn it into a simulated world, and put it under your own control.

Tasks

1 · Describe the rover

Write a **URDF** for a four-wheel rover: a chassis, four wheels, four joints. Sensible dimensions — roughly 0.6 m long, 0.4 m wide, wheels around 0.1 m radius.

Get the frame names right. `base_link` is the rover's body frame and the rest of ROS assumes it exists.

2 · See it before you simulate it

Load it with `robot_state_publisher` and view it in **RViz**. Use `joint_state_publisher_gui` to drag the wheels around by hand.

This step catches every geometry mistake in about ten seconds, and it costs nothing to run. Do it before you go near the simulator — debugging a bad URDF inside Gazebo is far harder than debugging it here.

3 · Understand the transform tree

Inspect it:

```
ros2 run tf2_tools view_frames
```

Every frame should connect back to `base_link` with no orphans. A broken TF tree is the single most common reason "everything looks fine but nothing works".

4 · Spawn it into Gazebo Harmonic

Put the rover in a world with some ground and a few obstacles.

> **Watch out:** Gazebo Harmonic is *not* Gazebo Classic. Most tutorials you'll find online are for Classic and use `gazebo_ros_pkgs` — those instructions will not work here. We use `ros_gz_sim`. If a tutorial mentions `gazebo_ros`, it's the wrong one.

5 · Bridge the topics

Gazebo's topics and ROS's topics are separate worlds. `ros_gz_bridge` connects them, one topic at a time, and you have to say which.

This is the concept that trips people up most this week. Nothing is broken when a topic doesn't appear in `ros2 topic list` — it just hasn't been bridged.

6 · Drive it by hand

Get a differential-drive plugin working so `/cmd_vel` moves the rover and `/odom` reports where it thinks it is. Then drive it with `teleop_twist_keyboard`.

7 · Drive it with your own code

Write a node that drives the rover in a **square** — 2 m per side — and returns roughly to where it started. Register the entry point as `square`.

Use `/odom` to decide when to stop and turn. Driving by timing alone works badly and you'll see exactly why: wheels slip, and the error compounds every corner.

In `MISSION_LOG.md`, record how far off the start point you ended up. That number is the reason the next mission exists.

8 · Mission log

What broke, what you worked out, what still doesn't make sense.

Deliverables

`src/task2/`	rover description, launch files, your driving node
screenshot	RViz showing the rover and its TF tree
`MISSION_LOG.md`	updated, including your final position error
video	under 90 s — rover driving its square in Gazebo

In the recording, **say out loud why the rover doesn't return exactly to its starting point.**

Checks

```
./tools/vanguard check
```

Stuck?

`./tools/vanguard doctor`, then the handbook, then open an Issue.

PART 2

Describing a robot

URDF, links, joints and transforms

10 — Describing a robot (URDF)

Time: 60–90 minutes. **By the end:** you have a four-wheel rover you can see and move in RViz.

What a URDF is

A **URDF** (Unified Robot Description Format) is an XML file that says what your robot is made of. Nothing more — it's a description, not a program.

But almost everything reads it: RViz to draw the robot, the simulator to give it mass and collision, the navigation stack to know how wide it is, the transform system to work out where each part is relative to every other part.

Get this file right and a lot of things start working for free. Get it wrong and you'll chase phantom bugs in code that isn't broken.

Two ideas: links and joints

- A **link** is a rigid body. The chassis is a link. Each wheel is a link.
- A **joint** connects two links and says how they may move relative to each other.

That's it. A robot is links connected by joints, forming a tree.

```
base_link (the chassis)
/ | | \
wheel_fl | | wheel_rr
wheel_fr wheel_rl
```

Every robot has exactly **one root link** and no loops.

Your first link

```
<?xml version="1.0"?>
<robot name="vanguard_rover">

 <link name="base_link">
 <visual>
 <geometry>
 <box size="0.6 0.4 0.15"/>
 </geometry>
 <material name="rover_grey">
 <color rgba="0.6 0.6 0.6 1.0"/>
 </material>
 </visual>
 </link>

</robot>
```

A link can have three parts, and they do different jobs:

Tag	Purpose	Who reads it
`<visual>`	what it looks like	RViz, the simulator's renderer
`<collision>`	what it bumps into	the physics engine
`<inertial>`	mass and how it's distributed	the physics engine

In RViz you only need `<visual>`. For simulation you need all three, and this catches people out: a link with no `<inertial>` may be ignored by physics entirely, so your rover falls through the floor or refuses to move.

`base_link` is not just a name we picked. Large parts of ROS assume a link called `base_link` exists and is the robot's body frame. Call it something else and tools will quietly fail to find your robot.

Adding a wheel

```
<link name="wheel_fl">
<visual>
<geometry>
<cylinder radius="0.1" length="0.05"/>
</geometry>
<material name="wheel_black">
<color rgba="0.1 0.1 0.1 1.0"/>
</material>
</visual>
</link>

<joint name="wheel_fl_joint" type="continuous">
<parent link="base_link"/>
<child link="wheel_fl"/>
<origin xyz="0.2 0.225 -0.05" rpy="-1.5708 0 0"/>
<axis xyz="0 0 1"/>
</joint>
```

Reading the joint:

- **`type="continuous"`** — rotates forever about one axis. That's a wheel. (Other types: `revolute` rotates but with limits; `fixed` doesn't move at all; `prismatic` slides.)
- **`<parent>`** / **`<child>`** — the joint's direction in the tree. The child moves with the parent.
- **`<origin xyz="...">`** — where the child sits relative to the parent, in metres. Here: 0.2 m forward, 0.225 m left, 0.05 m down.
- **`rpy="-1.5708 0 0"`** — roll, pitch, yaw in **radians**. A cylinder is created standing upright, so it needs rotating a quarter turn ($-\pi/2 \approx -1.5708$) to lie like a wheel.
- **`<axis xyz="0 0 1">`** — which axis it spins about, *in the child's own frame*.

> **Everything is metres and radians.** Type `90` where you meant `1.5708` and your wheel ends up pointing at the sky. Radians is a ROS-wide convention (REP-103) and it never changes.

Directions

Facing the same way the robot faces:

- **x** is forward
- **y** is left
- **z** is up

So the front-left wheel is at positive x, positive y. The rear-right is negative x, negative y. Work the four out on paper before typing them — it's much faster than discovering it in RViz.

Viewing it

Two nodes do this. **`robot_state_publisher`** reads the URDF and publishes where every link is. **`joint_state_publisher_gui`** gives you a slider per joint so you can move them by hand.

Write a launch file — `launch/display.launch.py`:

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
 pkg = get_package_share_directory('vanguard_rover')
 urdf = os.path.join(pkg, 'urdf', 'rover.urdf')

 with open(urdf, 'r') as f:
 robot_description = f.read()

 return LaunchDescription([
 Node(
 package='robot_state_publisher',
 executable='robot_state_publisher',
 parameters=[{'robot_description': robot_description}],
 ),
 Node(
 package='joint_state_publisher_gui',
 executable='joint_state_publisher_gui',
 ),
 Node(
 package='rviz2',
 executable='rviz2',
 ),
 ])
```

> `setup.py` **must install your URDF and launch files** or the launch file won't find them. Add to `data_files`: `python import os from glob import glob ... (os.path.join('share', package_name, 'urdf'), glob('urdf/*')), (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),` Forgetting this produces `FileNotFoundError` on a file you can plainly see on disk — because `ros2 launch` reads from `install/`, not from your source tree.

Then:

```
cd ~/vanguard_ws && colcon build --symlink-install && source install/setup.bash
ros2 launch vanguard_rover display.launch.py
```

In RViz: 1. Set **Fixed Frame** to `base_link` (it defaults to `map`, which doesn't exist yet — that's the "nothing is showing up" cause 90% of the time). 2. **Add → RobotModel**, then set its *Description Topic* to `/robot_description`. 3. **Add → TF** to see the frames.

Drag the sliders. The wheels should spin in place.

Check it before you trust it

```
check_urdf $(ros2 pkg prefix vanguard_rover)/share/vanguard_rover/urdf/rover.urdf
```

```
robot name is: vanguard_rover
---------- Successfully Parsed XML ---------------
root Link: base_link has 4 child(ren)
 child(1): wheel_fl
 ...
```

If it doesn't parse, nothing downstream will work. Fix it here.

If it went wrong

RViz is empty and RobotModel says "No transform from [x] to [map]" Fixed Frame is `map`. Change it to `base_link`.

Wheels in the wrong place or pointing oddly `rpy` is radians, and `<axis>` is in the *child's* frame, not the world's. Change one number at a time and watch.

`Error: Failed to build tree` / **`parent link not found`** A joint references a link that doesn't exist, or is spelled differently. Typos in link names are the usual cause. `check_urdf` names the offender.

Everything is at the origin, stacked on top of each other Your joints have no `<origin>`, so every child sits exactly on its parent.

Next: `11-tf.md`.

11 — Transforms (TF)

> **Time:** 30 minutes. **By the end:** you can read a transform tree and diagnose the most common silent failure in ROS.

The problem TF solves

Your rover's camera sees a rock 2 metres ahead. The navigation stack wants to know where that rock is *on the map*. The arm wants to know where it is relative to the gripper.

Same rock, three different answers — because each is measured from a different place. Converting between them by hand, continuously, as the robot moves, would be miserable and you'd get it wrong.

TF does that conversion for you. Every part of the robot publishes where it is relative to its parent, and TF composes those into an answer for any pair of frames at any point in time.

Frames

A **frame** is a coordinate system attached to something. Your URDF already created one per link — `base_link`, `wheel_fl`, and so on. Later you'll meet:

Frame	Meaning
`base_link`	the robot's body
`odom`	where the robot started, per its own wheel counting. Smooth, but drifts.
`map`	the world. Doesn't drift, but jumps when the robot corrects itself.

> **Why both `odom` and `map`?** Odometry is smooth but slowly wrong. Map-based localisation is right on average but jumps when it corrects. Controllers need smooth; planners need right. Keeping both means each gets what it needs. This is a genuinely elegant piece of design (REP-105) and it's worth understanding early.

Looking at the tree

With your robot running:

```
ros2 run tf2_tools view_frames
```

That listens for a few seconds and writes `frames.pdf` in your current directory. Open it. You should see `base_link` at the top with four wheels beneath it.

Ask about a specific pair:

```
ros2 run tf2_ros tf2_echo base_link wheel_fl
```

```
At time 0.0
- Translation: [0.200, 0.225, -0.050]
- Rotation: in Quaternion [-0.707, 0.000, 0.000, 0.707]
```

Those numbers should match the `<origin>` you wrote in the URDF. If they don't, your URDF isn't saying what you think it's saying.

The failure you will actually hit

TF failures are quiet. Nothing crashes; things just don't work.

Two disconnected trees. If `base_link` and `wheel_fl` appear as separate roots in `frames.pdf`, a joint is missing or misnamed. TF cannot answer any question across that gap, so anything needing both frames silently produces nothing.

Nothing publishing. `robot_state_publisher` not running means no transforms at all. RViz shows an empty world and a confusing error about a missing frame.

Time. Transforms are timestamped. Asking for one from before anything was published gives "*Lookup would require extrapolation into the past*". In simulation this usually means your `use_sim_time` setting is inconsistent between nodes.

Static vs dynamic

- **Static** — never changes. A camera bolted to the chassis. Published once.
- **Dynamic** — changes constantly. `odom` → `base_link` as the rover drives.

`robot_state_publisher` handles both from your URDF automatically: `fixed` joints become static, movable joints update from `/joint_states`.

The three commands

Question	Command
What's the tree?	`ros2 run tf2_tools view_frames`
Where is A relative to B?	`ros2 run tf2_ros tf2_echo A B`
What's on the wire?	`ros2 topic echo /tf`

When something involving positions misbehaves, look at the tree **first**. It's the cheapest check in robotics and it's right an unreasonable share of the time.

Next: `12-gazebo.md`.

PART 3

Into the simulator

Gazebo Harmonic and the ROS bridge

12 — Gazebo Harmonic and the bridge

Time: 90 minutes. **By the end:** your rover exists in a simulated world and moves when you tell it to.

Note: Read this before you search the internet

There are **two** Gazebos, and almost everything written online is about the wrong one.

	Gazebo Classic	Gazebo Harmonic
Status	end of life (Jan 2025)	current, what we use
Commands	`gazebo`, `rosrun gazebo_ros`	`gz sim`
ROS packages	`gazebo_ros_pkgs`, `gazebo_ros`	`ros_gz_sim`, `ros_gz_bridge`
Plugin names	`libgazebo_ros_diff_drive.so`	`gz-sim-diff-drive-system`

If a tutorial mentions `gazebo_ros`, `libgazebo_ros_*.so`, or tells you to run `gazebo`, it is for Classic and will not work here. This will happen constantly. Check before you spend an hour on it.

The tell: Classic tutorials are usually written for ROS Melodic, Noetic or Foxy. We're on **Jazzy**, which pairs with **Harmonic**.

Two separate worlds

This is the concept for this week, so slow down here.

Gazebo has its own messaging system — its own topics, its own message types — completely separate from ROS. They are two different programs that do not automatically understand each other.

```
Gazebo world ROS world
─────────────  ──────────

/model/rover/cmd_vel ←──┐  ┌──→ /cmd_vel
gz.msgs.Twist  │  │  geometry_msgs/msg/Twist
[ ros_gz_bridge ]
```

A **bridge** connects one topic to one topic, translating the message type. Nothing crosses unless you explicitly bridge it.

This explains the confusion you're about to have. You'll start the simulator, run `ros2 topic list`, and not see your robot's topics. Nothing is broken. They simply haven't been bridged yet.

Check your install

```
gz sim --version
```

```
Gazebo Sim, version 8.x.x
```

Version 8 is Harmonic. If this command isn't found:

```
sudo apt install -y ros-jazzy-ros-gz
```

Try an empty world:

```
gz sim empty.sdf
```

Press the ▶ **play** button at the bottom left. If the window opens and time advances, you're in business. Close it.

On slow machines add `-s` to run headless (no GUI) and `-v 4` for verbose logging.

A world to drive in

Worlds are **SDF** files — similar to URDF, but for whole scenes. Create `worlds/arena.sdf`:

```
<?xml version="1.0" ?>
<sdf version="1.10">
 <world name="arena">

 <plugin filename="gz-sim-physics-system"
 name="gz::sim::systems::Physics"/>
 <plugin filename="gz-sim-user-commands-system"
 name="gz::sim::systems::UserCommands"/>
 <plugin filename="gz-sim-scene-broadcaster-system"
 name="gz::sim::systems::SceneBroadcaster"/>

 <light type="directional" name="sun">
 <cast_shadows>true</cast_shadows>
 <pose>0 0 10 0 0 0</pose>
 <diffuse>0.8 0.8 0.8 1</diffuse>
 <direction>-0.5 0.1 -0.9</direction>
 </light>

 <model name="ground_plane">
 <static>true</static>
 <link name="link">
 <collision name="collision">
 <geometry><plane><normal>0 0 1</normal><size>100 100</size></plane></geometry>
 </collision>
 <visual name="visual">
 <geometry><plane><normal>0 0 1</normal><size>100 100</size></plane></geometry>
 <material><ambient>0.6 0.4 0.3 1</ambient><diffuse>0.6 0.4 0.3 1</diffuse></material>
 </visual>
 </link>
 </model>

 </world>
</sdf>
```

> Those three `<plugin>` lines are not optional. Without **Physics** nothing moves; without **SceneBroadcaster** you see an empty window even though the world loaded.

Add a few obstacle boxes yourself — you'll want them later.

Making the rover physical

Your earlier URDF only had `<visual>`. Physics needs mass and collision on **every** link:

```
 <link name="base_link">
 <visual>...</visual>

 <collision>
 <geometry><box size="0.6 0.4 0.15"/></geometry>
 </collision>

 <inertial>
 <mass value="10.0"/>
 <inertia ixx="0.15" ixy="0.0" ixz="0.0"
 iyy="0.32" iyz="0.0" izz="0.42"/>
 </inertial>
 </link>
```

The inertia numbers matter less than their being *present and non-zero*. For a box of mass m, width w, depth d, height h:

```
ixx = m(d² + h²)/12 iyy = m(w² + h²)/12 izz = m(w² + d²)/12
```

> **A link with no `<inertial>` is ignored by physics.** The classic symptoms: the rover falls through the ground, explodes on spawn, or simply refuses to move. If that happens, check every link has mass before you suspect anything else.

The drive plugin

Add this inside `<robot>`, at the end:

```
<gazebo>
<plugin filename="gz-sim-diff-drive-system"
name="gz::sim::systems::DiffDrive">
<left_joint>wheel_fl_joint</left_joint>
<left_joint>wheel_rl_joint</left_joint>
<right_joint>wheel_fr_joint</right_joint>
<right_joint>wheel_rr_joint</right_joint>
<wheel_separation>0.45</wheel_separation>
<wheel_radius>0.1</wheel_radius>
<odom_publish_frequency>50</odom_publish_frequency>
<topic>cmd_vel</topic>
<odom_topic>odom</odom_topic>
<frame_id>odom</frame_id>
<child_frame_id>base_link</child_frame_id>
<tf_topic>/tf</tf_topic>
</plugin>
</gazebo>
```

> Note: **`<tf_topic>` is easy to miss and breaks everything downstream.** Without it the plugin publishes the `/odom` *message* but never the `odom` → `base_link` **transform**. Your rover will drive, and `ros2 topic echo /odom` will look perfectly healthy — but RViz can't place the robot and anything needing to know where it is quietly fails. Bridge `/tf` as well (below), then check with:
> `bash ros2 run tf2_ros tf2_echo odom base_link`

Four wheels, two per side — that's **skid steering**, the same way a tank turns. Listing two `<left_joint>` entries drives both left wheels together.

`wheel_separation` is the distance between the left and right wheel *centres*. Get it wrong and the rover turns at the wrong rate — worth measuring against your URDF rather than guessing.

Spawning

`ros_gz_sim`'s `create` node puts a robot into a running world:

```
# Terminal 1
gz sim worlds/arena.sdf

# Terminal 2
ros2 run robot_state_publisher robot_state_publisher \
 --ros-args -p robot_description:="$(cat urdf/rover.urdf)"

# Terminal 3
ros2 run ros_gz_sim create -topic robot_description -z 0.2
```

`-z 0.2` drops it slightly above the ground so it settles rather than starting intersecting the floor.

Do this properly in a launch file once it works — three terminals gets old fast.

The bridge

Now connect the two worlds. The syntax is:

```
/topic_name@ros_message_type@gz_message_type
```

and the middle symbol sets direction:

Symbol	Direction
`@`	both ways
`[`	Gazebo → ROS (read a sensor)
`]`	ROS → Gazebo (send a command)

```
ros2 run ros_gz_bridge parameter_bridge \
 /cmd_vel@geometry_msgs/msg/Twist]gz.msgs.Twist \
 /odom@nav_msgs/msg/Odometry[gz.msgs.Odometry \
 /clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock \
 /tf@tf2_msgs/msg/TFMessage[gz.msgs.Pose_V
```

Read the first line as: *commands go from ROS into Gazebo*. The second: *odometry comes from Gazebo into ROS*.

Now:

```
ros2 topic list
```

`/cmd_vel` and `/odom` should be there. **They weren't before the bridge, and nothing was wrong.**

> **`/clock` matters more than it looks.** Simulated time doesn't run at wall-clock speed. Bridge `/clock` and set `use_sim_time: true` on your nodes, or timestamps disagree and TF starts complaining about extrapolation.

Drive it

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Keys are printed on screen. **That terminal must have focus** for keys to register.

Watch what it's doing:

```
ros2 topic echo /cmd_vel
ros2 topic echo /odom --once
ros2 topic hz /odom
```

If it went wrong

`gz: command not found` — `sudo apt install -y ros-jazzy-ros-gz`

World opens but is empty — missing the `SceneBroadcaster` plugin, or the file path is wrong. Run with `-v 4` and read the log.

Nothing moves, ever — press ▶ play. The simulator starts paused.

Rover falls through the floor / explodes on spawn — a link is missing `<inertial>` or `<collision>`, or a mass is zero.

`ros2 topic list` doesn't show `/cmd_vel` — not bridged, or the bridge exited. Look at the bridge's own terminal for errors.

Bridge runs but the rover ignores commands — the topic names don't line up. `ros2 topic echo /cmd_vel` proves messages are leaving ROS; then check `gz topic -l` to see what Gazebo is actually listening on.

Rover shudders, drifts or slides on its own — usually inertia values wildly inconsistent with the masses, or wheel friction. Try a heavier chassis first.

Everything is extremely slow — expected under software rendering (all Docker users). Try `gz sim -s` (headless) and watch in RViz instead.

Next: `13-closing-the-loop.md`.

PART 4

Driving with feedback

Closing the loop on odometry

13 — Closing the loop on odometry

Time: 60 minutes. **By the end:** your rover drives a square using feedback, and you understand why it doesn't come home exactly.

Open loop vs closed loop

The easy way to drive a square is by timing:

```
drive_forward(2.0) # for 4 seconds
turn_left(90) # for 1.8 seconds
# repeat 4 times
```

This is **open loop** — you issue commands and hope. It will not work well, and watching it fail is the point of this task.

Wheels slip. The simulator's timestep isn't perfectly uniform. The rover takes a moment to accelerate. Each error is small; there is nothing to correct them, so they accumulate. By the fourth corner you're metres and tens of degrees out.

Closed loop means looking at where you actually are and reacting:

```
while distance_travelled < 2.0:
 drive_forward()
```

Still imperfect — `/odom` is itself computed from wheel rotations, so it inherits wheel slip. But it corrects for timing and acceleration, and it's a large improvement for very little work.

Reading `/odom`

```
ros2 topic echo /odom --once
```

```
pose:
 pose:
 position: {x: 1.234, y: 0.056, z: 0.0}
 orientation: {x: 0.0, y: 0.0, z: 0.383, w: 0.924}
twist:
 twist:
 linear: {x: 0.5, y: 0.0, z: 0.0}
 angular: {z: 0.0, y: 0.0, z: 0.2}
```

- `pose.pose.position` — where it thinks it is, in the `odom` frame
- `pose.pose.orientation` — which way it's facing, as a **quaternion**
- `twist.twist` — how fast it's currently going

Quaternions, briefly

That orientation is four numbers, not an angle. Quaternions represent 3-D rotation without the failure modes of Euler angles.

You don't need to understand them today. You need to **convert one to a heading**:

```
import math

def yaw_from_quaternion(q):
 """Extract rotation about the vertical axis, in radians."""
 siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
 cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z)
 return math.atan2(siny_cosp, cosy_cosp)
```

`yaw` is your heading: 0 is along +x, π/2 is a quarter turn left. It wraps at ±π, which matters — see below.

Angle wrapping

The single most common bug in this task.

Heading runs from −π to +π and **wraps**. If you're at 3.0 rad and your target is −3.0 rad, the naive difference is 6.0 rad — nearly a full turn the wrong way. The real difference is about 0.28 rad the other way.

Always normalise:

```
def normalise(angle):
 """Wrap any angle into [-pi, pi]."""
 return math.atan2(math.sin(angle), math.cos(angle))

error = normalise(target_yaw - current_yaw)
```

Symptom if you skip this: the rover spins nearly all the way round instead of making a small correction, and only at *some* corners.

Structuring it

A state machine is the clean approach:

```
class SquareDriver(Node):
 def __init__(self):
 super().__init__('square_driver')
 self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
 self.sub = self.create_subscription(Odometry, '/odom', self.on_odom, 10)
 self.timer = self.create_timer(0.05, self.control_loop) # 20 Hz

 self.state = 'DRIVE' # DRIVE -> TURN -> DRIVE -> ... -> DONE
 self.pose = None
 self.leg = 0
 # record where this leg started, and what heading you want next

 def on_odom(self, msg):
 self.pose = msg.pose.pose # just store it

 def control_loop(self):
 if self.pose is None:
 return # no odometry yet — do nothing
 ...
```

Two things worth copying from that skeleton:

1. **The subscriber only stores data. The timer does the thinking.** Doing control inside the odometry callback couples your control rate to your sensor rate and makes everything harder to reason about.
2. **Guard against `None`.** Your node starts before the first `/odom` message arrives. Not checking is a guaranteed crash on startup.

Do it properly

- Stop **before** turning, and stop **before** driving. Trying to do both at once is a different and harder problem.
- Use a tolerance. `distance == 2.0` will never be exactly true with floats. Use `distance >= 2.0` or a small band.
- Send a zero `Twist` when you finish. A node that exits without stopping leaves the rover driving off — the last command stands.
- Log each leg with `get_logger().info()` so you can see what it thought it was doing.

The measurement

When it finishes, compare the final `/odom` position with where it started, and **write that error in your `MISSION_LOG.md`**:

```
Start: x=0.000 y=0.000
Finish: x=0.184 y=-0.092
Error: 0.206 m
```

Then try it again with more slip — drive faster, or make the turns sharper — and see how the error changes.

That number is the entire reason the next mission exists. A rover that navigates only by counting its own wheels gets steadily and confidently lost. Everything that fixes this involves looking at the outside world.

If it went wrong

Rover drives forever — your stop condition never becomes true. Log the distance each cycle and watch.

Spins wildly at corners — angle wrapping. See above.

Crashes with `'NoneType' object has no attribute 'position'` — the first control tick ran before any odometry arrived. Guard it.

Drives a rhombus, not a square — turns are overshooting or undershooting. Slow the angular velocity down as the heading error gets small, rather than turning at full speed until you're there.

Keeps moving after "DONE" — you never published a zero `Twist`.

Back to the Task 2 sheet.